2022 IEEE International Conference of Electron Devices Society Kolkata Chapter (EDKCON) | 978-1-6654-7205-0/22/$31.00 ©2022 IEEE | DOI: 10.1109/EDKCON56221.2022.10032954

Optimized Test Pattern Generation for Single Missing Gate Fault Detection in Reversible Arithmetic Circuits

Arindam Banerjee
Department of ECE
JIS College of Engineering
Kalyani, Nadia, West Bengal, India
Email: banerjee.arindam1@gmail.com

***Abstract*—** In this paper, optimized test pattern generation technique for single missing gate fault detection of reversible arithmetic circuits have been shown. Here the optimized test patterns, which are generated, are for reversible adder, subtractor and adder/ subtractor units. A generalized algorithm has been shown for generating the test patterns of different reversible arithmetic circuits.

Keywords*— *Reversible Logic, Arithmetic Circuits, Single Missing Gate Fault

I. Introduction

Reversible logic is an emerging field among the modern technologies. Reversible circuit testing is indispensable for the technological advancement. There are many types of faults which cause in reversible circuits. In conventional MOSFET or TTL technologies, stuck-at-faults occur whereas in reversible technology, missing gate faults occur [1-9]. In some recent works [1-3], new techniques have been shown for reversible circuit testing. As per [4], a missing gate fault is defined as complete disappearance of a reversible gate from a circuit. If a single gate disappears from a circuit, it is called a "single missing gate fault". If two or more consecutive gates disappear, the fault is called a "multiple missing gate fault" [4]. In some earlier works [4-9], few more fault detection techniques have been shown.

In this paper, the author tries to generate test set for detecting single missing gate fault (SMGF) for few reversible arithmetic circuits like adder, subtractor, adder-subtractor etc using some new pattern generation techniques.

II. Test Set Generation of Basic Reversible Gates

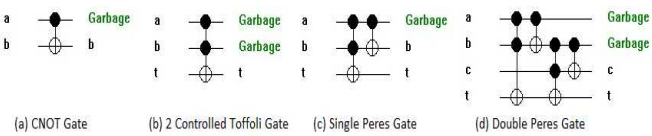

Fig. 1 Few Basic Reversible Gates

Fig. 1 shows the basic reversible gates. The first gate is CNOT or Controlled NOT gate. It has the characteristics matrix $CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$. If CNOT gate is missing then the characteristics matrix is basically an identity matrix i.e. $I = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$. If the two matrices are compared then at two positions (10 and 11), the difference is observed. Thus, the test pattern for the CNOT gate can be (10 or 11).

The second gate is 2 controlled Toffoli gate. Here in Fig. 1, the characteristics matrix for 3 input Toffoli gate is $Toffoli = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$. If the Toffoli gate is missing then the characteristics matrix is basically an identity matrix i.e. $I = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$. If the two matrices are compared then at two positions (110 and 111), the difference is observed. Thus the test pattern for the Toffoli gate can be (110 or 111).

The third gate is Peres gate which consists of one 3 input Toffoli gate (G_1) and one CNOT gate (G_2). The following table (Table-1) shows the input pattern with the input combinations to the two gates.

Table-1 Input Pattern with the input combinations to the Toffoli and CNOT gate in Single Peres Gate

a	b	t	G_1 (3 input Toffoli)	G_2 (CNOT)
0	0	0	000	00
0	0	1	001	00
0	1	0	010	01
0	1	1	011	01
1	0	0	100	10
1	0	1	101	10
1	**1**	**0**	**110**	**11**
1	**1**	**1**	**111**	**11**

From Table-1, it is obvious that at the input combinations 110 and 111, the two gates (Toffoli and CNOT) get the requisite test patterns. Thus the test patterns for the single Peres gate are (110 and 111). This is also verified from the following C code (Fig. 2 in Appendix Section).

The next gate is double Peres gate which is the combination of two Peres gates. Table-2 shows the input pattern with the input combinations to the four gates (2 Toffoli and 2 CNOT).

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:00:33 UTC from IEEE Xplore. Restrictions apply.

TABLE-2 INPUT PATTERN WITH THE INPUT COMBINATIONS TO THE TWO TOFFOLI AND TWO CNOT GATES IN DOUBLE PERES GATE

a	b	c	t	G_1 (First Toffoli)	G_2 (First CNOT)	G_3 (Second Toffoli)	G4 (Second CNOT)
0	0	0	0	000	00	000	00
0	0	0	1	001	00	000	00
0	0	1	0	000	00	010	01
0	0	1	1	001	00	011	01
0	1	0	0	010	01	100	10
0	1	0	1	011	01	101	10
0	**1**	**1**	**0**	**010**	**01**	**110**	**11**
0	1	1	1	011	01	111	11
1	0	0	0	100	10	000	00
1	0	0	1	101	10	001	00
1	0	1	0	100	10	010	01
1	0	1	1	101	10	011	01
1	1	0	0	110	11	001	00
1	1	0	1	111	11	000	00
1	**1**	**1**	**0**	**110**	**11**	**011**	**01**
1	1	1	1	111	11	010	01

From Table-2, it is obvious that at the input combinations 0110 and 1110, the four gates (two Toffoli and two CNOT) get the requisite test patterns. Thus the test patterns for the single Peres gate are (0110 and 1110). This is also verified from the following C code (Fig. 3). The following algorithm (Algorithm-1) describes the generalized technique for test pattern generation following C code shown in Fig. 2.

Algorithm-1:

Input: i=0, n, input_array[n], $Gate_i$_array, $test_i$_array;

Step-1: while(i<n)

```
        {
                Gate_i_array=input_array[i];
                i=i+1;
        }
```

Step-2: while(i<n)

```
        {
                Generate test_i_array for each ith gate;
                i=i+1;
        }
```

Step-3: for (i=0; i<n; i++)

```
        {
                if (Gate_i_array == test_i_array)
                {
                        print input_array[i];
                }
        }
```

III. TEST SET GENERATION OF HALF AND FULL ADDER

As shown in Fig. 4, the half adder circuit is basically a single Peres gate. As we know that the test pattern of a single Peres gate is (110 or 111) using Algorithm-1 and since here t = 0, thus the test pattern for the half adder is (110).

As shown in Fig. 5, the full adder circuit is basically a double Peres gate. Using Algorithm-1, we know that the test pattern of a double Peres gate is (0110 and 1110) and since here t = 0, thus the test pattern for the full adder is (0110 and 1110).

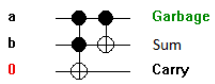

Fig. 4 Half Adder

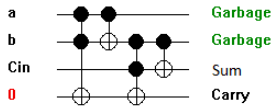

Fig. 5 Full Adder

IV. TEST SET GENERATION OF HALF AND FULL SUBTRACTOR

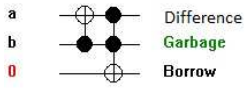

Fig. 6 Half Subtractor

TABLE-3 INPUT PATTERN WITH THE INPUT COMBINATIONS TO THE CNOT AND THE TOFFOLI GATES IN HALF SUBTRACTOR

a	b	t (Here 0)	G_1 (CNOT)	G_2 (3 input Toffoli gate)
0	0	0	00	000
0	0	1	00	001
0	**1**	**0**	**10**	**111**
0	**1**	**1**	**10**	**110**
1	0	0	01	100
1	0	1	01	101
1	1	0	11	010
1	1	1	11	011

For the half subtractor shown in Fig. 6, the test pattern to each of the two gates (one CNOT and one Toffoli gate) is shown in Table - 3 (here t=0 as shown in Fig. 6) . It is observed from Table-3 and using Algorithm-1 that (010 and 011) are the correct patterns to test the two gates. The third input (t) is 0 so the correct pattern is (010). This is also verified using the C code shown in Fig. 7 in Appendix Section.

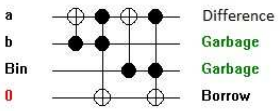

Fig. 8 Full Subtractor

For the full subtractor shown in Fig. 8, the test pattern to each of the four gates (two CNOT and two Toffoli gates) is shown in Table - 4. It is observed from Table – 4 and using Algorithm-1 that (0110, 0111, 1110 and 1111) are the correct patterns to test the two gates. The fourth input (t) is 0 so the correct pattern is (0110 and 1110). This is also verified using the C code shown in Fig. 9 in Appendix Section.

TABLE-4 INPUT PATTERN WITH THE INPUT COMBINATIONS TO THE TWO CNOT AND THE TWO TOFFOLI GATES IN FULL SUBTRACTOR

a	b	Bin	t	G_1 (First CNOT)	G_2 (First Toffoli)	G_3 (Second CNOT)	G_4 (Second Toffoli)
0	0	0	0	00	000	00	000
0	0	0	1	00	001	00	001
0	0	1	0	00	000	10	110
0	0	1	1	00	001	10	111
0	1	0	0	10	110	01	101
0	1	0	1	10	111	01	100
0	**1**	**1**	**0**	**10**	**110**	**11**	**011**
0	**1**	**1**	**1**	**10**	**111**	**11**	**010**
1	0	0	0	01	100	01	100
1	0	0	1	01	101	01	101
1	0	1	0	01	100	11	010
1	0	1	1	01	101	11	011
1	1	0	0	11	010	00	000
1	1	0	1	11	011	00	001
1	**1**	**1**	**0**	**11**	**010**	**10**	**110**
1	**1**	**1**	**1**	**11**	**011**	**10**	**111**

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:00:33 UTC from IEEE Xplore. Restrictions apply.

V. Test Set Generation of Adder- Subtractor

For the half adder-subtractor shown in Fig. 10, the input combination to each gate with the input pattern is shown in Table – 5 using Algorithm-1.

ctrl	a	b	t	G_1 (First CNOT)	G_2 (Toffoli)	G_3 (Second CNOT)	G_4 (Third CNOT)
0	0	0	0	00	000	00	00
0	0	0	1	00	001	00	00
0	0	1	0	00	010	01	01
0	0	1	1	00	011	01	01
0	1	0	0	01	100	10	01
0	1	0	1	01	101	10	01
0	1	1	0	01	110	11	00
0	1	1	1	01	111	11	00
1	0	0	0	10	100	10	11
1	0	0	1	10	101	10	11
1	**0**	**1**	**0**	**10**	**110**	**11**	**10**
1	**0**	**1**	**1**	**10**	**111**	**11**	**10**
1	1	0	0	11	000	00	10
1	1	0	1	11	001	00	10
1	1	1	0	11	010	01	11
1	1	1	1	11	011	01	11

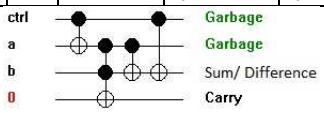

Fig. 10 Half Adder/ Subtractor

From Table - 5, it is observed that using the input patterns (1010 and 1011), all the gates (3 CNOT and one Toffoli) have been tested. Thus the correct pattern is (1010) since (t=0). It has been verified using the C code shown in Fig. 11 (in Appendix Section) using Algorithm-1.

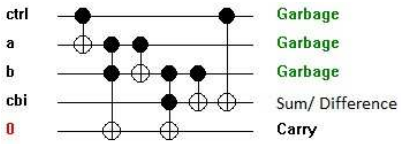

Fig. 12 Full Adder/ Subtractor

For the full adder/ subtractor shown in Fig. 12, the input combination to each gate with the input pattern is shown in Table – 6.

Table-6 Input Pattern with the input combinations to the four CNOT and the two Toffoli gates in Full Adder-Subtractor

ctrl	a	b	cbi	t	G_1 CNOT	G_2 Toffoli	G_3 CNOT	G_4 Toffoli	G_5 CNOT	G_6 CNOT
0	0	0	0	0	00	000	00	000	00	00
0	0	0	0	1	00	001	00	001	00	00
0	0	0	1	0	00	000	00	010	01	01
0	0	0	1	1	00	001	00	011	01	01
0	0	1	0	0	00	010	01	100	10	01
0	0	1	0	1	00	011	01	101	10	01
0	0	1	1	0	00	010	01	110	11	00
0	0	1	1	1	00	011	01	111	11	00
0	1	0	0	0	01	100	11	100	10	01
0	1	0	0	1	01	101	11	101	10	01
0	1	0	1	0	01	100	11	110	11	00
0	1	0	1	1	01	101	11	111	11	00
0	1	1	0	0	01	110	10	000	00	00
0	1	1	0	1	01	111	10	001	00	00
0	1	1	1	0	01	110	10	010	01	01
0	1	1	1	1	01	111	10	011	01	01
1	0	0	0	0	10	100	11	100	10	11
1	0	0	0	1	10	101	11	101	10	11
1	0	0	1	0	10	100	11	110	11	10
1	0	0	1	1	10	101	11	111	11	10
1	0	1	0	0	10	110	10	000	00	10
1	0	1	0	1	10	111	10	001	00	10
1	**0**	**1**	**1**	**0**	**10**	**110**	**10**	**010**	**01**	**11**
1	0	1	1	1	10	111	10	011	01	11
1	1	0	0	0	11	000	00	000	00	10
1	1	0	0	1	11	001	00	001	00	10
1	1	0	1	0	11	000	00	010	01	11
1	1	0	1	1	11	001	00	011	01	11
1	1	1	0	0	11	010	01	100	10	11
1	1	1	0	1	11	011	01	101	10	11
1	**1**	**1**	**1**	**0**	**11**	**010**	**01**	**110**	**11**	**10**
1	1	1	1	1	11	011	01	111	11	10

From Table - 6, it is observed that using the input patterns (10110 and 11110), all the gates (4 CNOT and two Toffoli) have been tested. It has been verified using the C code shown in Fig. 13 (in Appendix Section) using Algorithm-1.

VI. Computation Complexity Calculation

For a single bit adder or subtractor or adder-subtractor module, the test pattern is calculated one time. If n-bit module is tested then this pattern is same for each single bit module. Thus, if for a single bit module, the time taken by the processor is t unit then for n-bit module, the total time taken is $\frac{n \times t}{n} = t$ because the testing process is parallel for each single bit module. Therefore, for large value of n, the time complexity is unchanged and it is same as for the single bit module.

VII. Conclusion

The novelty of the paper was to optimize the number of test patterns to detect single missing gate faults in some particular class of reversible circuits like adder, subtractor, adder-subtractor etc. which were basically used for arithmetic operations. Using the test sets, each subcircuit of main circuit could be tested as missing or not. For each kind of circuit, the test set was different. For example, for reversible half adder, the test set was (110) and for full adder, the test set was (0110 and 1110). Similarly, for half subtractor, the test set was (010) and for full subtractor, the test set was (0110 and 1110). Similarly, for half adder-subtractor, the test set was (1010) and for the full adder-subtractor, the test set was (10110 and 11110). The test patterns were optimized using a generalized algorithm described in Algorithm-1. Moreover, for n-bit architecture, the test pattern was unique so that the time complexity was equal to the time complexity of the single bit architecture. It saved the time for single missing gate fault detection for large value of n.

Acknowledgment

The author is thankful to the Retired Professor Debesh Kumar Das, Department of CSE, Jadavpur University for his valuable suggestion.

References

[1] Joyati Mondal and Debesh Kumar Das, “A new online testing technique for reversible circuits”, IET Quantum Communication, 2022, pp. 50-59.

[2] Joyati Mondal, Dipak K. Kole, Hafizur Rahaman, Bhargab B. Bhattacharya, “DFT with Universal Test Set for All Missing Gate

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:00:33 UTC from IEEE Xplore. Restrictions apply.

Faults in Reversible Circuits", Journal of Circuits, Systems and Computers, vol. 31, no. 10, December, 2021.

[3] Joyati Mondal, Arighna Deb and Debesh Kumar Das, "An Efficient Design for Testability Approach of Reversible Logic Circuits", vol. 30, no. 6, October, 2020.

[4] H. M. Gaur, A. K. Singh and U. Ghanekar, "A Comprehensive and Comparative Study on Online Testability for Reversible Logic", Pertanika Journal Science and Technology, vol. 24, no. 2, pp. 245-271, 2016

[5] X. Fang-ying, C. Han-wu, W. Liu and L. Zhi-giang, "Fault Detection for Single and Multiple Missing Gate-Faults in Reversible Circuits", IEEE Congress on Evolutionary Computation, 2008.

[6] Hafizur Rahaman, Dipak K. Kole, Debesh K. Das and Bhargab B. Bhattacharya, "On the Detection of Missing-Gate Faults in Reversible Circuits by a Universal Test Set", Int. Conf. on VLSI Design, 4-8 January, 2008.

[7] Masoud Zamani and Mehdi B. Tahoori, "Online Missing/ Repeated Gate Faults Detection in Reversible Circuits," Int. Symp. on Defect and Fault Tolerance in VLSI and Nanotechnology Systems, 3-5 October, 2011.

[8] Hafizur Rahaman, Dipak K. Kole, Debesh K. Das and Bhargab B.Bhattacharya, "Fault diagnosis in reversible circuits under missing-gate fault model", Computers and Electrical Engineering, Elsevier, vol. 37, issue 4, July, 2011, pp. 475-485.

[9] Dipak K. Kole, Hafizur Rahaman, Debesh K. Das and Bhargab B.Bhattacharya, "Derivation of test set for detecting multiple missing-gate faults in reversible circuits", Computers and Electrical Engineering, Elsevier, vol. 29, issue 2, February, 2013, pp. 225-236.

APPENDIX

```c
#include<stdio.h>
#include<conio.h>
#include<math.h>

void main()
{
    int i,j,k,l,inp[8][3],outp[8][3],tof[8][3],cnot[8][2];
    for(i=0;i<8;i++)
    {
        k=i;
        for(j=0;j<3;j++)
        {
            inp[i][j]=k%2;
            k/=2;
        }
    }
    for(i=0;i<8;i++)
    {
        tof[i][2]=inp[i][2];
        tof[i][1]=inp[i][1];
        tof[i][0]=inp[i][0];
        cnot[i][1]=inp[i][2];
        cnot[i][0]=inp[i][1];
    }
    for(i=0;i<8;i++)
    {
        for(j=2;j>=0;j--)
        {
            printf("%d",inp[i][j]);
        }
        printf("\t");
        for(j=2;j>=0;j--)
        {
            printf("%d",tof[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot[i][j]);
        }
        printf("\n");
    }
    getch();
}
```

Fig. 2 C code describing the test pattern generation technique for single Peres gate

```c
#include<stdio.h>
#include<conio.h>
#include<math.h>
void main()
{
    int i,j,k,inp[16][4],outp[16][4],tof1[16][3],cnot1[16][2],tof2[16][3],cnot2[16][2];
    for(i=0;i<16;i++)
    {
        k=i;
        for(j=0;j<4;j++)
        {
            inp[i][j]=k%2;
            k/=2;
        }
    }
    for(i=0;i<16;i++)
    {
        outp[i][0]=(inp[i][3] & inp[i][2]) ^ inp[i][0];
        outp[i][2]=inp[i][3] ^ inp[i][2];
        outp[i][0]=(outp[i][2] & inp[i][3]) ^ outp[i][0];
        outp[i][1]=outp[i][2] ^ inp[i][1];
        outp[i][3]=inp[i][3];
        tof1[i][2]=inp[i][3];
        tof1[i][1]=inp[i][2];
        tof1[i][0]=inp[i][0];
        cnot1[i][1]=inp[i][3];
        cnot1[i][0]=inp[i][2];
        tof2[i][2]=outp[i][2];
        tof2[i][1]=inp[i][1];
        tof2[i][0]=(inp[i][3] & inp[i][2]) ^ inp[i][0];
        cnot2[i][1]=outp[i][2];
        cnot2[i][0]=inp[i][1];
    }
    for(i=0;i<16;i++)
    {
        for(j=3;j>=0;j--)
        {
            printf("%d",inp[i][j]);
        }
        printf("\t");
        for(j=2;j>=0;j--)
        {
            printf("%d",tof1[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot1[i][j]);
        }
        printf("\t");
        for(j=2;j>=0;j--)
        {
            printf("%d",tof2[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot2[i][j]);
        }
        printf("\n");
    }
    getch();
}
```

Fig. 3 C code describing the test pattern generation technique for double Peres gate

```c
#include<stdio.h>
#include<conio.h>
#include<math.h>

void main()
{
    int i,j,k,l,inp[8][3],outp[8][3],tof[8][3],cnot[8][2];
    for(i=0;i<8;i++)
    {
        k=i;
        for(j=0;j<3;j++)
        {
            inp[i][j]=k%2;
            k/=2;
        }
    }
    for(i=0;i<8;i++)
    {
        cnot[i][0]=inp[i][2];
        cnot[i][1]=inp[i][1];
        tof[i][2]=inp[i][1] ^ inp[i][2];
        tof[i][1]=inp[i][1];
        tof[i][0]=(tof[i][2] & tof[i][1]) ^ inp[i][0];
    }
    for(i=0;i<8;i++)
    {
        for(j=2;j>=0;j--)
        {
            printf("%d",inp[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot[i][j]);
        }
        printf("\t");
        for(j=2;j>=0;j--)
        {
            printf("%d",tof[i][j]);
        }
        printf("\n");
    }
    getch();
}
```

Fig. 7 C code describing the test pattern generation technique for half subtractor

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:00:33 UTC from IEEE Xplore. Restrictions apply.

```c
#include<stdio.h>
#include<conio.h>
#include<math.h>
void main()
{
    int i,j,k,inp[16][4],outp,tof1[16][3],cnot1[16][2],tof2[16][3],cnot2[16][2];
    for(i=0;i<16;i++)
    {
        k=i;
        for(j=0;j<4;j++)
        {
            inp[i][j]=k%2;
            k/=2;
        }
    }
    for(i=0;i<16;i++)
    {
        cnot1[i][1]=inp[i][2];
        cnot1[i][0]=inp[i][3];
        outp=((inp[i][3] ^ inp[i][1]) & inp[i][2]) ^ inp[i][0];
        tof1[i][2]=inp[i][3] ^ inp[i][2];
        tof1[i][1]=inp[i][2];
        tof1[i][0]=inp[i][0];
        cnot2[i][1]=inp[i][3];
        cnot2[i][0]=tof1[i][2];
        tof2[i][2]=tof1[i][2] ^ inp[i][1];
        tof2[i][1]=inp[i][3];
        tof2[i][0]=outp;
    }
    for(i=0;i<16;i++)
    {
        for(j=3;j>=0;j--)
        {
            printf("%d",inp[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot1[i][j]);
        }
        printf("\t");
        for(j=2;j>=0;j--)
        {
            printf("%d",tof1[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot2[i][j]);
        }
        printf("\t");
        for(j=2;j>=0;j--)
        {
            printf("%d",tof2[i][j]);
        }
        printf("\n");
    }
    getch();
}
```

Fig. 9 C code describing the test pattern generation technique for full subtractor

```c
#include<stdio.h>
#include<conio.h>
#include<math.h>
void main()
{
    int i,j,k,l,inp[16][4],outp[16][4],tof[16][3],cnot1[16][2],cnot2[16][2],cnot3[16][2];
    for(i=0;i<16;i++)
    {
        k=i;
        for(j=0;j<4;j++)
        {
            inp[i][j]=k%2;
            k/=2;
        }
    }
    for(i=0;i<16;i++)
    {
        cnot1[i][1]=inp[i][3];
        cnot1[i][0]=inp[i][2];
        tof[i][2]=inp[i][3] ^ inp[i][2];
        tof[i][1]=inp[i][1];
        tof[i][0]=inp[i][0];
        cnot2[i][1]=inp[i][3] ^ inp[i][2];
        cnot2[i][0]=inp[i][1];
        cnot3[i][1]=inp[i][3];
        cnot3[i][0]=(inp[i][3] ^ inp[i][2]) ^ inp[i][1];
    }
    for(i=0;i<16;i++)
    {
        for(j=3;j>=0;j--)
        {
            printf("%d",inp[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot1[i][j]);
        }
        printf("\t");
        for(j=2;j>=0;j--)
        {
            printf("%d",tof[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot2[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot3[i][j]);
        }
        printf("\n");
    }
    getch();
}
```

Fig. 11 C code describing the test pattern generation for half adder/ subtractor

```c
#include<stdio.h>
#include<conio.h>
#include<math.h>
void main()
{
    int i,j,k,inp[32][5],cnot1[32][3],tof1[32][3],cnot2[32][3],tof2[32][3],cnot3[32][3],cnot4[32][3];
    for(i=0;i<32;i++)
    {
        k=i;
        for(j=0;j<5;j++)
        {
            inp[i][j]=k%2;
            k/=2;
        }
    }
    for(i=0;i<32;i++)
    {
        cnot1[i][1]=inp[i][4];
        cnot1[i][0]=inp[i][3];
        tof1[i][2]=inp[i][4] ^ inp[i][3];
        tof1[i][1]=inp[i][2];
        tof1[i][0]=inp[i][0];
        cnot2[i][1]=inp[i][4] ^ inp[i][3];
        cnot2[i][0]=(inp[i][4] ^ inp[i][3]) ^ inp[i][2];
        tof2[i][2]=(inp[i][4] ^ inp[i][3]) ^ inp[i][2];
        tof2[i][1]=inp[i][1];
        tof2[i][0]=tof1[i][0];
        cnot3[i][1]=(inp[i][4] ^ inp[i][3]) ^ inp[i][2];
        cnot3[i][0]=inp[i][1];
        cnot4[i][1]=inp[i][4];
        cnot4[i][0]=((inp[i][4] ^ inp[i][3]) ^ inp[i][2]) ^ inp[i][1];
    }
    for(i=0;i<32;i++)
    {
        for(j=4;j>=0;j--)
        {
            printf("%d",inp[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot1[i][j]);
        }
        printf("\t");
        for(j=2;j>=0;j--)
        {
            printf("%d",tof1[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot2[i][j]);
        }
        printf("\t");
        for(j=2;j>=0;j--)
        {
            printf("%d",tof2[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot3[i][j]);
        }
        printf("\t");
        for(j=1;j>=0;j--)
        {
            printf("%d",cnot4[i][j]);
        }
        printf("\n");
    }
    getch();
}
```

Fig. 13 C code describing the test pattern generation technique for full adder/ subtractor

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:00:33 UTC from IEEE Xplore. Restrictions apply.